SWGL -- Animation overrides: give a character or an NPC its own animations

2026-10-13

What an animation override is

All the humanoid models of the game share one skeleton and one animation file, `models/players/_humanoid/_humanoid.gla`. Every character therefore moves the same way. An **animation override** lets a model (or a single NPC) use its own animations in place of some of these, **without rebuilding the big** `_humanoid.gla`. You make a small GLA that holds only the animations you want to change, plus a text file that names them. The game does the rest. It is **data only**: no code change.

How it works, in short:

1. At load time the renderer builds a *virtual* GLA: `_humanoid.gla` (with `_weapons.gla`, see below), then your GLA appended at the end.
2. The game reads your `.cfg` with the same rules as `animation.cfg`. Every animation you list **replaces the animation of the same name** of the base set. Every animation you do not list stays the stock one.
3. The character plays the new animations everywhere: in game, in the character menu, as player, as NPC, in saved games.

Only SP, only the **rd-vanilla**, **rd-rend2** and **rd-vulkan** renderers (the three SWGL renderers).

File (example)	Role	Required
`models/players/_myanims/_myanims.gla`	The animations: same skeleton as `_humanoid.gla`, only the frames you need.	Yes
`models/players/_myanims/_myanims.cfg` (or `animation.cfg`)	Name, first frame, length, loop, fps of each animation of the GLA.	Yes
`models/players/_myanims/animevents.cfg`	Sound / effect events of these animations.	No
`models/players/<model>/animoverride.cfg`	Makes a **model** use the override.	Only for the "model" way

1. The two ways to use an override

	Per model	Per instance
How	`animoverride.cfg` in the folder of the model	`animOverride` in the `.npc`, or the `animoverride` console command
Who gets it	Every character that uses this model, player and NPC	One entity
Lasts	Always	Until the map changes (console), or for the life of the NPC (`.npc`)

	Per model	Per instance
Best for	A character with a signature style (Vader, Malak...)	Testing, one-off NPCs, scripts

A per-instance override wins over the one of the model.

2. Make the animation GLA

2.1 The skeleton rule

The GLA must have **exactly the skeleton of** `_humanoid.gla`: the same number of bones, with the same names and the same parents, in the same order. The game compares them bone by bone. If one differs, the override is **not used** and the console prints:

```
GLA merge: models/players/_myanims/_myanims.gla does not have the skeleton of
models/players/_humanoid/_humanoid.gla, or too many frames
```

Export from your 3D tool using the `_humanoid` skeleton (the stock `.glm` / `.gla` workflow is the same as for any Jedi Academy animation: Blender or 3ds Max + carcass / assimilate). Name the GLA's internal animation file `models/players/_humanoid/_humanoid`: this is what the ragdoll and a few other checks look for. Name the files with the usual `.gla` extension.

2.2 The frame budget

Frames of the three parts are stacked in one file: `_humanoid.gla` (21 384 frames), `_weapons.gla`, then your GLA. A frame number must stay under **65 535** in total. That leaves room for a few thousand frames. If the sum is too big, the override is refused with the message above.

2.3 Where the folder goes

The folder name is the name of the override. The convention is a leading underscore, `_myanims`, so that it cannot be mistaken for a character model: the GLA is `models/players/_myanims/_myanims.gla`. The game also accepts the folder without the underscore (`myanims/myanims.gla`); with both present, the underscore folder wins.

3. The animation file

`models/players/_myanims/_myanims.cfg` is read first. If it does not exist the game reads `models/players/_myanims/animation.cfg`. The format is the one of the stock `animation.cfg`:

```
// name              start   length   loop   fps
BOTH_STAND1          0       3        0      1
BOTH_STAND2          8       1        -1     60
BOTH_SABERFAST_STANCE 8      1        -1     60
BOTH_MELEE_COMBO_1   10      32       -1     50
```

Column	Meaning
name	An animation name of the game (`BOTH_STAND1` , `TORSO_RAISEWEAP1` , `LEGS_WALK1` ...). The list is `animTable` in `bg_panimate.cpp` / `anims.h` . An unknown name is printed in red (`WARNING: Unknown token ... in ...`) and ignored. `ROOT` is skipped without a message.
start	First frame **in your GLA**. Count from 0. The game adds the offset of your GLA in the merged file for you.
length	Number of frames.
loop	Frame to loop back to, relative to the start. `-1` = no loop (play once and stop on the last frame, or whatever the game does for this animation).
fps	Frames per second. A negative value plays backwards.

Rules:

- **Only list what you changed.** An animation that is not in your file keeps the `_humanoid` one.
- An animation can have a single frame (a pose, e.g. a weapon stance): `length 1` .
- Several names can point at the same frames (see `BOTH_STAND2` , `BOTH_STAND2IDLE1` ... on the same frame in the stock overrides).
- Comments with `//` . Tabs or spaces.

Events (optional)

`models/players/_myanims/animevents.cfg` uses the exact syntax of the stock `animevents.cfg` (`UPPEREVENTS` / `LOWEREVENTS` blocks: footsteps, sounds, effects, saber swings). The frame of an event is the frame **inside the animation** (0 = first frame), not a frame of the GLA. Events of the same animation and frame stomp those of the base set.

4. Use it on a model: `animoverride.cfg`

In the folder of the model (the one with `model.glm`), create `animoverride.cfg` :

```
overridename _myanims
```

One line, one key. The value is the **exact folder name** of your override (`_myanims` , with its underscore if it has one). Conditions:

- The model must use the **`_humanoid` skeleton** (its `.glm` points at `models/players/_humanoid/_humanoid`). A model with another skeleton ignores the file.
- `models/players/_myanims/_myanims.gla` must exist. If not: the renderer prints `models/players/<model>/animoverride.cfg: models/players/_myanims/_myanims.gla is missing` and the model uses the stock animations.
- A missing `overridename` key prints `no "overridename" key` .

All the characters whose `.cha` variant has `model <model>` now use it, including in the character menu preview.

To give two models the same style, point both `animoverride.cfg` at the same folder.

5. Use it on one NPC or one entity

In an `.npc`

```
elween
{
        playerModel               elween
        animOverride      _myanims
        ...
}
```

`animOverride` goes with the other fields of the NPC (see the Character guide, section 1.4). If the override cannot be applied, the console prints `NPC elween: animOverride _myanims is not used`.

With the console

Command	Effect
`animoverride`	Prints the animation set of the player and the usage.
`animoverride <name>`	Forces the override `<name>` on the player. `<name>` is the folder: `_myanims` or `myanims`, both work.
`animoverride <name> <targetname>`	Same, on the NPC whose `targetname` is `<targetname>`.
`animoverride default`	Forces the stock `_humanoid.gla`, even if the model has an `animoverride.cfg`.
`animoverride none`	Removes the forced override. The model uses its own `animoverride.cfg` again, if it has one.

The current animations restart at once, with the new frames. The setting lasts until the map changes. It is **kept in saved games** (the set of animations of the entity is saved and restored on load).

6. Weapons: `_weapons.gla`

`models/weapons2/_animations/_weapons.gla` (with its `_weapons.cfg` and `animevents.cfg`) holds the weapon poses and is appended to **every** `_humanoid*.gla`, hence to every override too. You do not need to copy weapon animations into your GLA. If you want a character to hold a weapon differently, list the same animation name in your `.cfg` (for example `TORSO_PLX_AIM`) and your version replaces the stock one for this character only.

7. Pack and test

1. Put the files in a `Z_<name>.pk3` (paths start at the game root: `models/players/_myanims/...`, `models/players/<model>/animoverride.cfg`) in `GameData\SWGL\`. It loads after the `SWGL_*.pk3` and can replace a stock override (same path).
2. Quick test without touching a model: start a map and type `animoverride _myanims` in the console. If nothing changes, read the console. A character that stays in T-pose or freezes means a wrong `.cfg` (check the names and the frame numbers).
3. Set `developer 1` to read the merge lines: `GLA merge: animation override _myanims: frames 21800 to 22100 come from models/players/_myanims/_myanims.gla`.

4. Once it works, add `animoverride.cfg` to the model, or `animOverride` to the `.npc`.

8. Checklist and common mistakes

Symptom	Cause
`animoverride: <name> is not used, see the console`	The GLA is missing, its skeleton differs from `_humanoid`, or the total is over 65 535 frames.
Nothing changes with `animoverride.cfg`	The model does not use the `_humanoid` skeleton, the folder name in `overridename` is misspelt, or the key is missing.
Some animations are the stock ones	Expected: only the names listed in your `.cfg` are replaced.
`WARNING: Unknown token X in ...cfg`	The animation name is not in the game's animation table (spelling).
Animation plays the wrong frames	`start` is counted in your own GLA from 0, not in the merged file. Do not add any offset yourself.
Event plays at the wrong time	In `animevents.cfg` the frame is relative to the start of the animation.
Override lost after a map change (console)	Expected: use `animoverride.cfg` or `animOverride` for something permanent.
Ragdoll or death animation odd	The internal animation file name of the GLA is not `models/players/_humanoid/_humanoid`.
Same name used by a stock override	Names must be unique among the packs: the pk3 loaded last wins.

Code references: `rd-common/mdx_merge.h` (merge rules and constants), `rd-common/tr_glamerge.cpp` (virtual GLA, reading of `animoverride.cfg`), `game/NPC_stats.cpp` (`G_ParseMergedAnimations`, `G_SetAnimOverride`, `G_RestoreAnimOverrides`, `animOverride` key), `game/g_cmds.cpp` (`Cmd_AnimOverride_f`).

Stock overrides to copy from, in `SWGL_Animations.pk3`: `_malak` (one-file example, mostly poses), `_kotor`, `_clones`, `_dvader`, `_jedi`, `_jedi2`, `_lanakin`, `_oldben`, `_palpatine`, `_shaak`, `_sidious`, `_vader`, `_zombie` and others. Each has its GLA, its `.cfg`, and the matching `animoverride.cfg` in the model folder.